Министерство науки и высшего образования РФ
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра Информационные технологии и автоматизированные системы

Базы данных

Лабораторная работа № 4

Тема: «Реализация команд DDL, DML и соединений на веб-странице»

Выполнил: студент
группы РИС-25-2б
Ильинов Ф. М.
Проверил: доцент кафедры ИТАС
Петренко А.А.

г. Пермь – 2026

СОДЕРЖАНИЕ

1 Цель работы 4

2 Задачи работы 5

3 Этапы выполнения 6

3.1 Архитектура веб-приложения и стек вызовов 7

3.1.1 Веб-сервер и маршрутизация запросов 7

3.1.2 Архитектура обработчиков и обработка ошибок 9

3.1.3 Движок представления и шаблонизация 11

3.1.4 Пул соединений и взаимодействие с PostgreSQL 13

3.1.5 Механизм безопасной сборки SQL-запросов 14

3.2 Динамическое получение метаданных PostgreSQL и просмотр таблиц 16

3.2.1 Чтение системного каталога information_schema 16

3.2.2 GET /tables 17

3.3 Реализация команд DDL 21

3.3.1 Создание таблиц: POST /ddl/create 21

3.3.2 Модификация схемы: POST /ddl/alter 25

3.3.3 Очистка и удаление таблиц: TRUNCATE и DROP 27

3.4 Реализация команд DML 29

3.4.1 Пакетная вставка записей: POST /dml/insert 29

3.4.2 Модификация данных: стек вызова POST /dml/update 32

3.4.3 Удаление данных: стек вызова POST /dml/delete 34

3.5 Реализация выборок и соединений (DQL) 36

3.5.1 Параметризованная выборка: POST /query/select 36

3.5.2 Внутреннее соединение таблиц: POST /query/join 41

3.6 Механизм сброса и начальной инициализации схемы 43

3.6.1 Начальная инициализация схемы базы данных 43
3.6.2 Сброс POST /reset 45
3.7 Результаты работы и снимки экрана приложения 47
Заключение 53
Список использованных источников 54

1 Цель работы

Реализовать веб-приложение для взаимодействия с СУБД PostgreSQL, обеспечивающее выполнение полного жизненного цикла операций с реляционной базой данных:

а) Команды DDL: создание, модификация, очистка и удаление таблиц;

б) Команды DML: вставка, обновление и удаление;

в) Команды SELECT и INNER JOIN ... ON.

2 Задачи работы

В соответствии с целью были поставлены следующие задачи:

а) Разработать локальный веб-сервер на языке Golang (версия 1.27) с использованием стандартной библиотеки `net/http`;

б) Реализовать подключение к СУБД PostgreSQL с использованием пула соединений `pgx`;

в) Разработать механизм безопасной трансляции параметров пользовательских форм в команды SQL с экранированием;

г) Реализовать стек вызовов для команд DDL;

д) Реализовать стек вызовов для команд DML;

е) Реализовать стек вызовов для выборок и соединений: построитель запросов SELECT и оператор соединения INNER JOIN по ключевым полям;

ж) Разработать веб-интерфейс на основе Server-Side Rendering (пакет `html/template`) с выводом результатов и текста выполненного SQL-запроса на страницу браузера.

3 Этапы выполнения

Для выполнения поставленных задач были сформулированы следующие технологические требования к инструментарию:

– Поддержка реляционной модели данных и стандарта SQL в СУБД PostgreSQL [1, 2];

– Поддержка сетевого протокола передачи гипертекста HTTP [3];

– Возможность отрисовки форм, кнопок и динамических таблиц в веб-интерфейсе.

Для реализации клиентского интерфейса был выбран язык разметки HTML [4] совместно с библиотекой стилей Pico CSS [5], обеспечивающей лаконичное визуальное оформление без необходимости написания сложного клиентского JavaScript-кода.

В качестве языка серверной части выбран Golang версии 1.27 [6]. Для взаимодействия с СУБД PostgreSQL задействован драйвер и пул соединений `github.com/jackc/pgx` [7].

Сервер функционирует по принципу Server-Side Rendering (SSR). Все страницы собираются на стороне сервера с помощью встроенного шаблонизатора `html/template` и возвращаются браузеру в виде готового HTML-документа.

3.1 Архитектура веб-приложения и стек вызовов

Веб-приложение построено по многоуровневой схеме. Прохождение любого пользовательского действия организовано в виде строгого сквозного стека вызовов:

а) клиентский уровень: веб-страница браузера (HTML-формы с параметрами операций DDL, DML или DQL);

б) уровень веб-сервера и маршрутизации: маршрутизатор стандартной библиотеки Golang (версия 1.27), диспетчеризация по HTTP-методу и пути, распаковка параметров формы;

в) уровень сборщика запросов (Query Builder): экранирование идентификаторов PostgreSQL двойными кавычками (`quoteIdent`) и строковых литералов одинарными кавычками (`quoteLiteral`);

г) уровень СУБД PostgreSQL: выполнение запроса через пул соединений `pgx`, опрос системного каталога `information_schema`;

д) уровень представления (View Renderer): динамическое сканирование кортежей и отрисовка HTML-шаблона с отображением текста выполненного SQL-запроса и полученных данных.

3.1.1 Веб-сервер и маршрутизация запросов

Инициализация сетевого мультиплексора и регистрация маршрутов выполняются в функции `main` (листинг 1).

```
76  mux := http.NewServeMux()
77  mux.Handle("GET /css/", http.StripPrefix("/css/", cssFS))
78
79  mux.HandleFunc("GET /{$}", func(w http.ResponseWriter, r *http.Request) {
80      http.Redirect(w, r, "/tables", http.StatusSeeOther)
81  })
82
83  mux.Handle("GET /tables", HandleTables(views, pool, sqlMgr))
84  mux.Handle("GET /tables/{name}", HandleTables(views, pool, sqlMgr))
85
86  mux.HandleFunc("GET /ddl", func(w http.ResponseWriter, r *http.Request) {
87      http.Redirect(w, r, "/ddl/create", http.StatusSeeOther)
88  })
89  mux.Handle("GET /ddl/{op}", HandleDDL(views, pool, sqlMgr))
90  mux.Handle("POST /ddl/create", CreateTableHandler(views, pool, sqlMgr))
91  mux.Handle("POST /ddl/alter", AlterTableHandler(views, pool, sqlMgr))
92  mux.Handle("POST /ddl/truncate", TruncateTableHandler(views, pool, sqlMgr))
93  mux.Handle("POST /ddl/drop", DropTableHandler(views, pool, sqlMgr))
94
95  mux.HandleFunc("GET /dml", func(w http.ResponseWriter, r *http.Request) {
96      http.Redirect(w, r, "/dml/insert", http.StatusSeeOther)
97  })
98  mux.Handle("GET /dml/{op}", HandleDML(views, pool, sqlMgr))
99  mux.Handle("POST /dml/insert", InsertHandler(views, pool, sqlMgr))
100 mux.Handle("POST /dml/update", UpdateHandler(views, pool, sqlMgr))
101 mux.Handle("POST /dml/delete", DeleteHandler(views, pool, sqlMgr))
102
103 mux.Handle("GET /select", HandleSelect(views, pool, sqlMgr))
104 mux.Handle("GET /select/{table}", HandleSelect(views, pool, sqlMgr))
105 mux.Handle("POST /query/select", SelectHandler(views, pool, sqlMgr))
106
107 mux.Handle("GET /join", HandleJoin(views, pool, sqlMgr))
108 mux.Handle("POST /query/join", JoinHandler(views, pool, sqlMgr))
109
```

Листинг 1 — Маршрутизация HTTP-запросов DDL, DML и DQL в `main.go`

В строке 79 шаблон `"GET /{$}"` задает строгое соответствие корневому пути и выполняет перенаправление на страницу таблиц `/tables`. Для остальных операций предусмотрено четкое разделение:

— `GET`-запросы отображают страницы и формы (например, `/ddl/{op}`, `/dml/{op}`, `/select`);

— `POST`-запросы принимают данные форм и запускают сборку и выполнение соответствующих SQL-команд (`/ddl/create`, `/dml/insert`, `/query/select` и др.).

3.1.2 Архитектура обработчиков и обработка ошибок

Для унификации обработки ошибок используется тип-адаптер `AppHandler` (листинг 2), представляющий собой функцию, возвращающую ошибку.

```go
type AppError struct {
    Err     error
    Message string
    Code    int
}

func (e *AppError) Error() string {
    if e.Message != "" {
        return e.Message
    }
    if e.Err != nil {
        return e.Err.Error()
    }
    return "Internal Server Error"
}

func (e *AppError) Unwrap() error {
    return e.Err
}

type AppHandler func(w http.ResponseWriter, r *http.Request) error

func (fn AppHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    if err := fn(w, r); err != nil {
        var appErr *AppError
        if errors.As(err, &appErr) {
            slog.Error("request error",
                slog.String("path", r.URL.Path),
                slog.String("message", appErr.Message),
                slog.Any("err", appErr.Err),
            )
            code := appErr.Code
            if code == 0 {
                code = http.StatusInternalServerError
            }
            http.Error(w, appErr.Message, code)
            return
        }

        slog.Error("unhandled server error", slog.String("path",
        ↪  r.URL.Path), slog.Any("err", err))
        http.Error(w, "Internal server error: "+err.Error(),
        ↪  http.StatusInternalServerError)
    }
}
```

Листинг 2 — Структура `AppError` и адаптер обработчиков `AppHandler`

Метод `ServeHTTP` перехватывает ошибки выполнения. Если возвращена ошибка `AppError`, фиксируется запись в логе и клиенту отправляется соответствующий HTTP-статус. При возникновении ошибки на уровне СУБД форма рендерится повторно со статусом `400 Bad Request`, сохраняя введенные пользователем значения и подсвечивая сообщение сервера PostgreSQL и проблемный SQL-запрос.

3.1.3 Движок представления и шаблонизация

Генерация веб-интерфейса реализована на стороне сервера с помощью встроенного пакета `html/template` и обертки `ViewRenderer` (листинг 3).

```
14 type ViewRenderer struct {
15     templates map[string]*template.Template
16 }
17
18 func NewViewRenderer(funcMap template.FuncMap) (*ViewRenderer, error) {
19     layout, err := template.New("layout.html").Funcs(funcMap).ParseFiles(
       ↪  "templates/layout.html")
20     if err != nil {
21         return nil, fmt.Errorf("parse layout.html: %w", err)
22     }
23
24     pages := []string{"tables", "ddl", "dml", "select", "join"}
25     tmpls := make(map[string]*template.Template, len(pages))
26     for _, p := range pages {
27         t, err := layout.Clone()
28         if err != nil {
29             return nil, fmt.Errorf("clone layout for %s: %w", p, err)
30         }
31         path := filepath.Join("templates", p+".html")
32         t, err = t.ParseFiles(path)
33         if err != nil {
34             return nil, fmt.Errorf("parse %s: %w", path, err)
35         }
36         tmpls[p] = t
37     }
38     return &ViewRenderer{templates: tmpls}, nil
39 }
```

Листинг 3 — Инициализация и рендеринг шаблонов в `views.go`

При запуске сервера компилируется базовый шаблон `layout.html`, после чего методом `Clone()` создаются изолированные копии для каждой рабочей страницы (`tables`, `ddl`, `dml`, `select`, `join`). Для динамических вычислений в разметке зарегистрированы специальные функции (листинг 4).

```
funcMap := template.FuncMap{
    "add": func(a, b int) int { return a + b },
    "seq": func(n int) []int {
        res := make([]int, n)
        for i := range res {
            res[i] = i
        }
        return res
    },
    "eqStr": func(a, b string) bool { return a == b },
    "getRowVal": func(rows []map[string]string, rIdx int, colName string)
    ↪ string {
        if rIdx >= 0 && rIdx < len(rows) && rows[rIdx] != nil {
            return rows[rIdx][colName]
        }
        return ""
    },
}
```

Листинг 4 — Пользовательские функции шаблонизатора `funcMap`

Функции `seq` и `add` используются для циклической генерации динамических строк формы (например, добавление заданного количества колонок в DDL или кортежей в DML). Функция `getRowVal` обеспечивает безопасное извлечение сохраненных значений полей при повторной отрисовке после ошибок.

Базовый шаблон `layout.html` (листинг 5) содержит навигацию, кнопку быстрого сброса БД и блок отображения сгенерированного SQL-запроса. Для предотвращения выхода широких таблиц за границы страницы контейнерам `figure` назначен стиль горизонтальной прокрутки `overflow-x: auto`.

```
16 <main class="container">
17     <header>
18         <article>
19             <strong>DB Laboratory Work №4</strong><br>
20             <small>RIS-25-2b Ilinov F. M.</small>
21             <form action="/reset" method="post">
22                 <button type="submit">Reset</button>
23             </form>
24         </article>
25         <article>
26             <a role="button" href="/tables">SHOW</a>
27            <a role="button" href="/ddl/create">Data Definition Language</a>
28             <a role="button" href="/dml/insert">Data Manipulation
               ↪  Language</a>
29             <a role="button" href="/select">SELECT</a>
30             <a role="button" href="/join">INNER JOIN</a>
31         </article>
32     </header>
33
34     {{ if eqStr .Status "error" }}
35     <article>
36         <header><strong>ERROR:</strong> {{ .Message }}</header>
37         {{ if .SQL }}
38         <pre><code>{{ .SQL }}</code></pre>
39         {{ end }}
40     </article>
41     {{ end }}
42
43     {{ block "content" . }}
44     {{ end }}
45
46 </main>
```

Листинг 5 — Разметка контейнера и блок аудита SQL в `layout.html`

В строках 34–41 размещен блок уведомления об ошибках: если сервер СУБД возвращает ошибку, точный текст сформированного запроса выводится в теге `<pre><code>`.

3.1.4 Пул соединений и взаимодействие с PostgreSQL

Подключение к СУБД осуществляется в функции `main` через пул соединений библиотеки `pgx` (листинг 6).

```
pool, err := pgx.NewConnPool(pgx.ConnPoolConfig{
    Host:           dbHost,
    Port:           dbPort,
    Database:       dbName,
    User:           dbUser,
    Password:       dbPassword,
    MaxConnections: 10,
})
if err != nil {
    slog.Error("cannot connect to postgres", slog.Any("err", err))
    os.Exit(1)
}
defer pool.Close()
```

Листинг 6 — Инициализация пула соединений pgx

Пул соединений повторно использует открытые TCP-сессии к PostgreSQL, снижая накладные расходы на постоянную аутентификацию и установление соединения при каждом HTTP-запросе.

3.1.5 Механизм безопасной сборки SQL-запросов

Поскольку имена таблиц и столбцов передаются из веб-интерфейса динамически, прямое использование позиционных плейсхолдеров PostgreSQL (`$1, $2`) для них неприменимо. Для защиты от внедрения вредоносного SQL-кода (SQL Injection) в файле `db.go` реализованы специализированные функции экранирования (листинг 7).

```
func quoteIdent(name string) string {
    return `"` + strings.ReplaceAll(name, `"`, `""`) + `"`
}

func quoteLiteral(val string) string {
    return `'` + strings.ReplaceAll(val, `'`, `''`) + `'`
}
```

Листинг 7 — Функции экранирования идентификаторов и литералов

Функция `quoteIdent` экранирует идентификаторы PostgreSQL (имена таблиц, колонок) согласно стандарту: имя оборачивается в двойные кавыч-

ки, а внутренние кавычки удваиваются (""). Функция `quoteLiteral` обрамляет пользовательские строковые константы в одинарные кавычки с удвоением внутренних одинарных кавычек.

Однако часть параметров форм нельзя экранировать ни как идентификатор, ни как литерал: это типы данных столбцов, операторы сравнения в условиях WHERE и правила `ON DELETE`/`ON UPDATE` внешних ключей. Они подставляются в текст запроса как есть, поэтому перед сборкой SQL проверяются по белым спискам допустимых значений в файле `validate.go` (листинг 8).

```
// Типы данных, операторы сравнения и правила внешних ключей нельзя
// экранировать как идентификатор или литерал, поэтому они подставляются
// в SQL только после проверки по белому списку.
var (
    allowedColumnTypes = []string{"INTEGER", "TEXT", "BOOLEAN", "DATE"}
    allowedCompareOps  = []string{"=", "<>", "<", ">", "<=", ">=", "LIKE",
    ↪  "ILIKE", "IS NULL", "IS NOT NULL"}
    allowedSelectOps   = slices.Concat(allowedCompareOps, []string{"IN",
    ↪  "NOT IN"})
    allowedFKActions   = []string{"", "NO ACTION", "CASCADE", "SET NULL",
    ↪  "RESTRICT"}
)

func checkAllowed(kind, value string, allowed []string) error {
    if slices.Contains(allowed, strings.ToUpper(strings.TrimSpace(value)))
    ↪  {
        return nil
    }
    return fmt.Errorf("invalid %s: %q", kind, value)
}
```

Листинг 8 — Белые списки допустимых значений и функция проверки `checkAllowed`

Если значение отсутствует в списке, запрос не отправляется в СУБД, а пользователю выводится сообщение об ошибке. Так, попытка передать в поле оператора строку `= '' OR 1=1; DROP TABLE departments; --` отклоняется с сообщением `invalid WHERE operator`.

3.2 Динамическое получение метаданных PostgreSQL и просмотр таблиц

В отличие от статических приложений, где структура таблиц зафиксирована в структурах данных языка программирования, разрабатываемый вебсервис должен уметь взаимодействовать с произвольными таблицами, создаваемыми пользователем в рантайме. Для этого реализовано динамическое получение метаданных СУБД.

3.2.1 Чтение системного каталога information_schema

Получение перечня пользовательских таблиц и описания их столбцов выполняется посредством запросов к стандартным системным представлениям `information_schema` СУБД PostgreSQL.

Для получения списка базовых таблиц публичной схемы используется SQL-запрос из файла `get_tables.sql` (листинг 9).

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE'
ORDER BY table_name;
```

Листинг 9 — Запрос списка пользовательских таблиц из `information_schema.tables`

Условие `table_schema = 'public'` исключает системные таблицы СУБД, а предикат `table_type = 'BASE TABLE'` фильтрует виртуальные представления (VIEW).

Для чтения структуры выбранной таблицы используется параметризованный запрос из файла `get_columns.sql` (листинг 10).

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = $1
ORDER BY ordinal_position;
```

Листинг 10 — Запрос описания колонок таблицы из `information_schema.columns`

Как и в предыдущем запросе, условие `table_schema = 'public'` ограничивает поиск публичной схемой, чтобы одноименные таблицы из других схем не дублировали список колонок. Запрос возвращает имена атрибутов и их типы данных (`data_type`), упорядоченные по физическому положению в таблице (`ordinal_position`).

3.2.2 GET /tables

Просмотр содержимого таблиц инициируется запросом по маршруту `GET /tables` (для вывода всех таблиц) или `GET /tables/{name}` (для выбранной таблицы). Логика обработки сосредоточена в функции `HandleTables` (листинг 11).

```
9  func HandleTables(views *ViewRenderer, pool *pgx.ConnPool, sql *SQLManager)
   ↪  AppHandler {
10     return func(w http.ResponseWriter, r *http.Request) error {
11         base, err := newBasePage(pool, sql)
12         if err != nil {
13             return &AppError{Err: err, Message: "Database error: " +
                ↪  err.Error(), Code: http.StatusInternalServerError}
14         }
15
16         data := TablesPageData{BasePage: base}
17         tableName := r.PathValue("name")
18         if tableName == "" {
19             tableName = r.URL.Query().Get("table")
20         }
21
22         if tableName == "" {
23             data.ShowViewAll = true
24             if all, err := getAllTablesData(pool, sql); err == nil {
25                 data.AllTables = all
26             }
27         } else {
28             data.SelectedTable = tableName
29             td, err := getTableData(pool, sql, tableName)
30             if err != nil {
31                 data.Status = "error"
32                 data.Message = "Table not found: " + err.Error()
33                 return views.RenderStatus(w, "tables", http.StatusNotFound,
                    ↪  data)
34             }
35             data.CurrentTable = td
36         }
37
38         return views.Render(w, "tables", data)
39     }
```

Листинг 11 — Обработчик страницы просмотра таблиц в `handlers_tables.go`

В строке 11 вызывается функция `newBasePage`, наполняющая навигационный выпадающий список актуальным перечнем таблиц из каталога СУБД. Если конкретная таблица не выбрана, вызывается `getAllTablesData`, опрашивающая каждую таблицу по очереди.

Поскольку схема возвращаемых данных заранее неизвестна, в файле db.go реализована функция scanRowsToTableData (листинг 12).

```
376 func scanRowsToTableData(rows *pgx.Rows, tableName string) (*TableData,
    ↪  error) {
377     fields := rows.FieldDescriptions()
378     colNames := make([]string, len(fields))
379     for i, f := range fields {
380         colNames[i] = f.Name
381     }
382 
383     var dataRows [][]any
384     for rows.Next() {
385         vals, err := rows.Values()
386         if err != nil {
387             return nil, err
388         }
389         for i, v := range vals {
390             if t, ok := v.(time.Time); ok {
391                 vals[i] = t.Format("2006-01-02")
392             }
393         }
394         dataRows = append(dataRows, vals)
395     }
396     if err := rows.Err(); err != nil {
397         return nil, err
398     }
399 
400     return &TableData{
401         Name:    tableName,
402         Columns: colNames,
403         Rows:    dataRows,
404     }, nil
405 }
```

Листинг 12 — Динамическое чтение метаданных выборки и значений кортежей

Метод rows.FieldDescriptions() извлекает названия колонок из заголовка ответа PostgreSQL, а метод rows.Values() считывает кортеж значений в обобщенный слайс []any. Значения даты и времени (time.Time) приводятся к стандартизированному строковому виду "YYYY-MM-DD".

Полученная структура `TableData` передается в шаблон `tables.html` (листинг 13), где выполняется отрисовка HTML-таблицы.

```
<figure>
    <table>
        <thead>
            <tr>
                {{ range .Columns }}
                <th>{{ . }}</th>
                {{ end }}
            </tr>
        </thead>
        <tbody>
            {{ range .Rows }}
            <tr>
                {{ range . }}
                <td>{{ . }}</td>
                {{ end }}
            </tr>
            {{ else }}
            <tr><td colspan="{{ len .Columns }}"><em>Empty</em></td></tr>
            {{ end }}
        </tbody>
    </table>
</figure>
```

Листинг 13 — Отрисовка динамической таблицы в `templates/tables.html`

Цикл `{{ range .Columns }}` генерирует заголовки таблицы (`<th>`), а вложенный цикл `{{ range .Rows }}` итерирует строки и ячейки (`<td>`).

3.3 Реализация команд DDL

Команды Data Definition Language отвечают за создание, модификацию и удаление объектов базы данных. В веб-приложении операции DDL сгруппированы по маршруту `/ddl/{op}` со следующими режимами: `create`, `alter`, `truncate`, `drop`.

3.3.1 Создание таблиц: POST /ddl/create

Процесс создания таблицы разделен на два логических шага:

а) первый шаг: пользователь указывает желаемое количество столбцов будущей таблицы;

б) второй шаг: динамически строится форма с полями конфигурации каждого столбца: наименование, тип данных PostgreSQL (`INTEGER`, `TEXT`, `BOOLEAN`, `DATE`), признак допустимости NULL значений (`Nullable`), выбор первичного ключа (`PRIMARY KEY`) и параметры внешнего ключа (`FOREIGN KEY`).

Фрагмент формы второго шага приведен в листинге 14.

```
<tbody>
    {{ range $i, $col := .CreateColumns }}
    <tr>
        <td>{{ add $i 1 }}</td>
        <td><input type="text" name="col_name_{{ $i }}" value="{{ $col.Name }}" placeholder="column_{{ add $i 1 }}" required></td>
        <td>
            <select name="col_type_{{ $i }}">
                <option value="INTEGER" {{ if eqStr $col.DataType "INTEGER" }}selected{{ end }}>INTEGER</option>
                <option value="TEXT" {{ if eqStr $col.DataType "TEXT" }}selected{{ end }}>TEXT</option>
                <option value="BOOLEAN" {{ if eqStr $col.DataType "BOOLEAN" }}selected{{ end }}>BOOLEAN</option>
                <option value="DATE" {{ if eqStr $col.DataType "DATE" }}selected{{ end }}>DATE</option>
            </select>
        </td>
        <td>
            <input type="checkbox" name="col_nullable_{{ $i }}" value="1" {{ if $col.IsNullable }}checked{{ end }}>
        </td>
```

Листинг 14 — Динамическая форма настройки колонок таблицы в `templates/ddl.html`

При отправке формы запрос поступает на эндпоинт `POST /ddl/create`, где запускается парсер параметров `parseCreateColumns` (листинг 15).

```
34 func parseCreateColumns(r *http.Request, sql *SQLManager, colCount int,
   ↪  tableName string) ([]ColumnDef, []string) {
35     pkIndex, pkErr := strconv.Atoi(r.FormValue("pk_index"))
36     hasPK := pkErr == nil && pkIndex >= 0 && pkIndex < colCount
37
38     parsedCols := make([]ColumnDef, colCount)
39     var colDefs []string
40     var pkColName string
41
42     for i := 0; i < colCount; i++ {
43         cName := strings.TrimSpace(r.FormValue(fmt.Sprintf("col_name_%d",
           ↪  i)))
44         cType := strings.TrimSpace(r.FormValue(fmt.Sprintf("col_type_%d",
           ↪  i)))
45         if cType == "" {
46             cType = "TEXT"
47         }
48         isNullable := r.FormValue(fmt.Sprintf("col_nullable_%d", i)) == "1"
49         fkTable := strings.TrimSpace(r.FormValue(fmt.Sprintf("fk_table_%d",
           ↪  i)))
50         fkColRaw := strings.TrimSpace(r.FormValue(fmt.Sprintf("fk_col_%d",
           ↪  i)))
51         fkSelfIdx := -1
52
```

Листинг 15 — Парсинг полей столбцов и первичного ключа в `handlers_ddl.go`

Особое внимание уделено поддержке самоссылающихся внешних ключей (префикс `__self__`), когда столбец ссылается на первичный ключ этой же создаваемой таблицы. Разбор ссылки внешнего ключа приведен в листинге 16.

```
53  if strings.HasPrefix(fkColRaw, "__self__:") {
54      fkTable = "__self__"
55      if idx, err := strconv.Atoi(strings.TrimPrefix(fkColRaw, "__self__:"));
        ↪  err == nil {
56          fkSelfIdx = idx
57      }
58  } else if strings.Contains(fkColRaw, ":") {
59      parts := strings.SplitN(fkColRaw, ":", 2)
60      fkTable = parts[0]
61      fkColRaw = parts[1]
62  } else if fkSelfStr :=
    ↪  strings.TrimSpace(r.FormValue(fmt.Sprintf("fk_self_col_%d", i)));
    ↪  fkSelfStr != "" {
63      if idx, err := strconv.Atoi(fkSelfStr); err == nil {
64          fkSelfIdx = idx
65      }
66  }
67
68  if fkTable == "__self__" && fkSelfIdx == -1 {
69      if idx, err := strconv.Atoi(fkColRaw); err == nil {
70          fkSelfIdx = idx
71      }
72  }
```

Листинг 16 — Разбор ссылок внешних ключей в `handlers_ddl.go`

После формирования списка столбцов обработчик `CreateTableHandler` проверяет типы данных и правила внешних ключей функцией `checkCreateColumns`, затем формирует итоговый SQL-запрос и передает его СУБД (листинг 17).

```
219 if tableName == "" {
220     return renderErr("Table name cannot be empty", "")
221 }
222 if len(colDefs) == 0 {
223     return renderErr("At least one column must be specified", "")
224 }
225 if err := checkCreateColumns(parsedCols); err != nil {
226     return renderErr(err.Error(), "")
227 }
228
229 querySQL, err := sql.CreateTable(tableName, colDefs)
230 if err != nil {
231     return renderErr(fmt.Sprintf("Failed to render create table query: %v",
      ↪  err), "")
232 }
233 if _, err := pool.Exec(querySQL); err != nil {
234     return renderErr(fmt.Sprintf("Failed to create table (CREATE TABLE):
      ↪  %v", err), querySQL)
235 }
236
237 redirectSuccess(w, r, fmt.Sprintf("/tables/%s", url.PathEscape(tableName)))
238 return nil
```

Листинг 17 — Сборка и выполнение запроса `CREATE TABLE`

В случае успешного исполнения запроса клиент перенаправляется на страницу созданной таблицы `/tables/{name}`. При ошибке форма отображается повторно с сохранением введенных данных и выводом сообщения сервера СУБД.

3.3.2 Модификация схемы: POST /ddl/alter

Операция изменения структуры таблицы реализуется обработчиком `AlterTableHandler` (листинг 18).

```
270 if table == "" || effectiveColName == "" {
271     return renderErr("Specify table and column name", "")
272 }
273 
274 if action == "add" || action == "alter_type" {
275     if colType == "" {
276         colType = "TEXT"
277     }
278     if err := checkAllowed("column type", colType, allowedColumnTypes); err
      ↪  != nil {
279         return renderErr(err.Error(), "")
280     }
281 }
282 
283 var querySQL string
284 var err error
285 switch action {
286 case "add":
287     querySQL, err = sql.AlterAdd(table, effectiveColName, colType)
288 case "alter_type":
289     querySQL, err = sql.AlterType(table, effectiveColName, colType)
290 case "drop":
291     querySQL, err = sql.AlterDrop(table, effectiveColName)
292 default:
293     return renderErr("Unknown ALTER action", "")
294 }
295 
296 if err != nil {
297     return renderErr(fmt.Sprintf("Failed to render ALTER TABLE query: %v",
      ↪  err), "")
298 }
299 
300 if _, err := pool.Exec(querySQL); err != nil {
301     return renderErr(fmt.Sprintf("Failed to execute ALTER TABLE: %v", err),
      ↪  querySQL)
302 }
303 
304 redirectSuccess(w, r, fmt.Sprintf("/tables/%s", url.PathEscape(table)))
305 return nil
```

Листинг 18 — Сборка и исполнение команд `ALTER TABLE`

Для действий добавления и изменения типа столбца тип данных предварительно проверяется по белому списку (строки 274–281). Поддерживаются три сценария:

а) добавление нового столбца;

б) изменение типа существующего столбца;

в) удаление столбца.

3.3.3 Очистка и удаление таблиц: TRUNCATE и DROP

Для быстрого удаления всех записей таблицы без разрушения ее схемы используется обработчик `TruncateTableHandler` (листинг 19).

```
309 func TruncateTableHandler(views *ViewRenderer, pool *pgx.ConnPool, sql
    ↪  *SQLManager) AppHandler {
310     return func(w http.ResponseWriter, r *http.Request) error {
311         if err := r.ParseForm(); err != nil {
312             return &AppError{Err: err, Message: "Invalid form data", Code:
                ↪  http.StatusBadRequest}
313         }
314 
315         table := strings.TrimSpace(r.FormValue("table"))
316         if table == "" {
317             return renderDDLError(w, views, pool, sql, "truncate", 1, "",
                ↪  "Select a table to truncate", "", nil)
318         }
319 
320         querySQL, err := sql.TruncateTable(table)
321         if err != nil {
322             return renderDDLError(w, views, pool, sql, "truncate", 1,
                ↪  table, fmt.Sprintf("Failed to render TRUNCATE query: %v",
                ↪  err), "", nil)
323         }
324         if _, err := pool.Exec(querySQL); err != nil {
325             return renderDDLError(w, views, pool, sql, "truncate", 1,
                ↪  table, fmt.Sprintf("Failed to execute TRUNCATE: %v", err),
                ↪  querySQL, nil)
326         }
327 
328         redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
            ↪  url.PathEscape(table)))
329         return nil
330     }
331 }
```

Листинг 19 — Сборка и исполнение команды `TRUNCATE TABLE`

Удаление сущности целиком выполняется обработчиком `DropTableHandler` (листинг 20).

```
333 func DropTableHandler(views *ViewRenderer, pool *pgx.ConnPool, sql
    ↪  *SQLManager) AppHandler {
334     return func(w http.ResponseWriter, r *http.Request) error {
335         if err := r.ParseForm(); err != nil {
336             return &AppError{Err: err, Message: "Invalid form data", Code:
                ↪  http.StatusBadRequest}
337         }
338 
339         table := strings.TrimSpace(r.FormValue("table"))
340         if table == "" {
341             return renderDDLError(w, views, pool, sql, "drop", 1, "",
                ↪  "Select a table to drop", "", nil)
342         }
343 
344         querySQL, err := sql.DropTable(table)
345         if err != nil {
346             return renderDDLError(w, views, pool, sql, "drop", 1, table,
                ↪  fmt.Sprintf("Failed to render DROP TABLE query: %v", err),
                ↪  "", nil)
347         }
348         if _, err := pool.Exec(querySQL); err != nil {
349             return renderDDLError(w, views, pool, sql, "drop", 1, table,
                ↪  fmt.Sprintf("Failed to execute DROP TABLE: %v", err),
                ↪  querySQL, nil)
350         }
351 
352         redirectSuccess(w, r, "/tables")
353         return nil
354     }
355 }
```

Листинг 20 — Сборка и исполнение команды `DROP TABLE`

Указание ключевого слова `CASCADE` предотвращает блокировку операции СУБД в случаях, когда на удаляемую таблицу ссылаются внешние ключи других отношений.

3.4 Реализация команд DML

Команды Data Manipulation Language предназначены для добавления, изменения и удаления кортежей. В приложении поддерживаются три базовые операции: `INSERT INTO`, `UPDATE` и `DELETE`.

3.4.1 Пакетная вставка записей: POST /dml/insert

Добавление кортежей организовано в два шага:

а) первый шаг: пользователь выбирает целевую таблицу из выпадающего списка и указывает количество добавляемых строк;

б) второй шаг: сервер генерирует табличную сетку ввода, где каждый столбец соответствует реальному атрибуту таблицы из `information_schema`, а строки нумеруются функцией `seq`.

Фрагмент генерации сетки ввода приведен в листинге 21.

```
<figure>
    <table>
        <thead>
            <tr>
                <th>#</th>
                {{ range .TableColumns }}
                <th>
                    {{ .Name }} {{ .DataType }}

                </th>
                {{ end }}
            </tr>
        </thead>
        <tbody>
            {{ range $r := seq .DMLRowCount }}
            <tr>
                <td><strong>{{ add $r 1 }}</strong></td>
                {{ range $.TableColumns }}
                <td>
                    <input type="text" name="val_{{ $r }}_{{ .Name }}"
                    ↪  value="{{ getRowVal $.InsertRows $r .Name }}"
                    ↪  placeholder="{{ .Name }}">
                </td>
                {{ end }}
            </tr>
            {{ end }}
        </tbody>
    </table>
</figure>
```

Листинг 21 — Генерация динамической сетки ввода кортежей в `templates/dml.html`

При отправке данных обработчик `InsertHandler` вызывает функцию `parseInsertRows` (листинг 22), формирующую списки значений.

```
func parseInsertRows(r *http.Request, cols []string, rowCount int)
↪  ([]map[string]string, []string) {
    insertRows := make([]map[string]string, rowCount)
    var rowTuples []string

    for rIdx := 0; rIdx < rowCount; rIdx++ {
        rowMap := make(map[string]string, len(cols))
        var valLiterals []string
        for _, c := range cols {
            val := r.FormValue(fmt.Sprintf("val_%d_%s", rIdx, c))
            rowMap[c] = val
            if val == "" {
                valLiterals = append(valLiterals, "NULL")
            } else {
                valLiterals = append(valLiterals, quoteLiteral(val))
            }
        }
        insertRows[rIdx] = rowMap
        rowTuples = append(rowTuples, "("+strings.Join(valLiterals, ",
        ↪  ")+")")
    }
    return insertRows, rowTuples
}
```

Листинг 22 — Парсинг многострочных кортежей и экранирование в `handlers_dml.go`

Если значение ячейки оставлено пустым, оно преобразуется в ключевое слово `NULL`. Непустые значения экранируются функцией `quoteLiteral`. В результате формируется пакетный SQL-запрос с перечислением нескольких кортежей через запятую (листинг 23).

```
148         querySQL, err := sql.Insert(table, quotedCols, rowTuples)
149         if err != nil {
150             return renderErr(fmt.Sprintf("Failed to render INSERT INTO
                ↪  query: %v", err), "")
151         }
152 
153         if _, err := pool.Exec(querySQL); err != nil {
154             return renderErr(fmt.Sprintf("Failed to execute INSERT INTO:
                ↪  %v", err), querySQL)
155         }
156 
157         redirectSuccess(w, r, fmt.Sprintf("/tables/%s",
            ↪  url.PathEscape(table)))
158         return nil
159     }
160 }
```

Листинг 23 — Сборка и выполнение пакетного запроса `INSERT INTO`

3.4.2 Модификация данных: стек вызова POST /dml/update

Изменение данных реализуется обработчиком `UpdateHandler` (листинг 24).

```
185 if table == "" || setCol == "" {
186     return renderErr("Specify table and column to update", "")
187 }
188 if err := checkWhereOp(whereOp); err != nil {
189     return renderErr(err.Error(), "")
190 }
191 
192 whereClause := sql.SimpleWhere(whereCol, whereOp, whereVal)
193 querySQL, err := sql.Update(table, setCol, setVal, whereClause)
194 if err != nil {
195     return renderErr(fmt.Sprintf("Failed to render UPDATE query: %v", err),
      ↪  "")
196 }
197 
198 if _, err := pool.Exec(querySQL); err != nil {
199     return renderErr(fmt.Sprintf("Failed to execute UPDATE: %v", err),
      ↪  querySQL)
200 }
201 
202 redirectSuccess(w, r, fmt.Sprintf("/tables/%s", url.PathEscape(table)))
203 return nil
```

Листинг 24 — Конструирование и выполнение команды `UPDATE`

Пользователь указывает имя таблицы, целевую колонку, новое строковое значение, а также условие фильтрации. Оператор сравнения сначала проверяется функцией `checkWhereOp` по белому списку, после чего условие WHERE конструируется вспомогательной функцией `SimpleWhere` (листинг 25).

```
184 func (m *SQLManager) SimpleWhere(col, op, val string) string {
185     if col == "" || op == "" {
186         return ""
187     }
188     res, err := m.Render("where_simple.sql", map[string]any{
189         "Col": col,
190         "Op":  op,
191         "Val": val,
192     })
193     if err != nil {
194         return ""
195     }
196     return " " + strings.TrimSpace(res)
197 }
```

Листинг 25 — Конструирование предиката WHERE в `db.go`

3.4.3 Удаление данных: стек вызова POST /dml/delete

Удаление строк выполняется обработчиком `DeleteHandler` (листинг 26).

```
226 if table == "" {
227     return renderErr("Select a table", "")
228 }
229 if err := checkWhereOp(whereOp); err != nil {
230     return renderErr(err.Error(), "")
231 }
232 
233 whereClause := sql.SimpleWhere(whereCol, whereOp, whereVal)
234 querySQL, err := sql.Delete(table, whereClause)
235 if err != nil {
236     return renderErr(fmt.Sprintf("Failed to render DELETE query: %v", err),
      ↪  "")
237 }
238 
239 if _, err := pool.Exec(querySQL); err != nil {
240     return renderErr(fmt.Sprintf("Failed to execute DELETE: %v", err),
      ↪  querySQL)
241 }
242 
243 redirectSuccess(w, r, fmt.Sprintf("/tables/%s", url.PathEscape(table)))
244 return nil
```

Листинг 26 — Конструирование и выполнение команды `DELETE FROM`

3.5 Реализация выборок и соединений (DQL)

В приложении представлены два ключевых механизма: `SELECT` и `INNER JOIN`.

3.5.1 Параметризованная выборка: POST /query/select

Конструктор выборки `/select` позволяет пользователю интерактивно настраивать три компонента SQL-запроса:

а) проекция колонок;

б) предикаты фильтрации WHERE;

в) сортировка ORDER BY.

Фрагмент формы настройки выборки представлен в листинге 27.

```
{{ range .SelectColumns }}
<tr>
    <td><strong>{{ .Name }}</strong> <small>({{ .DataType }})</small></td>
    <td>
        <input type="checkbox" name="inc_{{ .Name }}" value="1" checked>
    </td>
    <td>
        <input type="checkbox" name="where_en_{{ .Name }}" value="1">
    </td>
    <td>
        <select name="where_op_{{ .Name }}">
            <option value="=">=</option>
            <option value="<>">&lt;&gt;</option>
            <option value="<">&lt;</option>
            <option value=">">&gt;</option>
            <option value="<=">&lt;=</option>
            <option value=">=">&gt;=</option>
            <option value="LIKE">LIKE (%val%)</option>
            <option value="ILIKE">ILIKE (%val%)</option>
            <option value="IN">IN (comma-separated list)</option>
            <option value="NOT IN">NOT IN (comma-separated list)</option>
            <option value="IS NULL">IS NULL</option>
            <option value="IS NOT NULL">IS NOT NULL</option>
        </select>
    </td>
    <td>
        <input type="text" name="where_val_{{ .Name }}" placeholder="Filter
        ↪ value">
    </td>
</tr>
```

Листинг 27 — Интерактивный конструктор выборки в `templates/select.html`

Обработка запроса выполняется в функции `SelectHandler`. Сначала для каждой отмеченной колонки считывается оператор и проверяется по белому списку `allowedSelectOps` (листинг 28).

```
var whereParts []string
for _, col := range cols {
    if r.FormValue("where_en_"+col.Name) == "1" {
        op := strings.TrimSpace(r.FormValue("where_op_" + col.Name))
        rawVal := strings.TrimSpace(r.FormValue("where_val_" + col.Name))
        if err := checkAllowed("WHERE operator", op, allowedSelectOps); err
        ↪  != nil {
            data.Status = "error"
            data.Message = err.Error()
            return views.Render(w, "select", data)
        }
```

Листинг 28 — Проверка оператора фильтрации в `handlers_query.go`

Затем в зависимости от оператора собирается предикат фильтрации (листинг 29). Значения экранируются функцией `quoteLiteral`, а для `LIKE` и `ILIKE` значение автоматически обрамляется символами `%`.

```
        switch op {
        case "IS NULL", "IS NOT NULL":
            part := sql.WhereNull(col.Name, op)
            whereParts = append(whereParts, part)
        case "IN", "NOT IN":
            items := strings.Split(rawVal, ",")
            var quotedItems []string
            for _, it := range items {
                if trimmed := strings.TrimSpace(it); trimmed != "" {
                    quotedItems = append(quotedItems,
                     ↪  quoteLiteral(trimmed))
                }
            }
            if len(quotedItems) > 0 {
                part := sql.WhereIn(col.Name, op, quotedItems)
                whereParts = append(whereParts, part)
            }
        case "LIKE", "ILIKE":
            part := sql.WhereOp(col.Name, op, "%"+rawVal+"%")
            whereParts = append(whereParts, part)
        default:
            part := sql.WhereOp(col.Name, op, rawVal)
            whereParts = append(whereParts, part)
        }
    }
}
```

Листинг 29 — Сборка условий фильтрации WHERE в `handlers_query.go`

Затем добавляется сортировка, запрос собирается и выполняется (листинг 30). Направление сортировки допускает только значения `ASC` и `DESC`.

```
orderByCol := strings.TrimSpace(r.FormValue("order_by"))
orderDir := strings.TrimSpace(r.FormValue("order_dir"))
if orderDir != "DESC" {
    orderDir = "ASC"
}
orderClause := ""
if orderByCol != "" && orderByCol != "none" {
    clause := sql.OrderBy(orderByCol, orderDir)
    orderClause = " " + clause
}

querySQL, err := sql.Select(table, selectList, whereClause, orderClause)
if err != nil {
    data.Status = "error"
    data.Message = "Failed to render query: " + err.Error()
    return views.Render(w, "select", data)
}

rows, err := pool.Query(querySQL)
if err != nil {
    data.Status = "error"
    data.Message = "Failed to execute SELECT: " + err.Error()
    data.SQL = querySQL
    return views.Render(w, "select", data)
}
defer rows.Close()

tableData, err := scanRowsToTableData(rows, table)
if err != nil {
    data.Status = "error"
    data.Message = "Failed to read query results: " + err.Error()
    data.SQL = querySQL
    return views.Render(w, "select", data)
}

data.QueryResult = tableData
return views.Render(w, "select", data)
```

Листинг 30 — Сортировка, сборка и выполнение запроса `SELECT` в `handlers_query.go`

Результат запроса считывается методом `pool.Query(querySQL)`, оборачивается в структуру `TableData` и отображается пользователю.

3.5.2 Внутреннее соединение таблиц: POST /query/join

Соединение таблиц по маршруту `/join` демонстрирует операцию внутреннего соединения (`INNER JOIN`). В качестве демонстрационной схемы используются таблицы `lab2_employee_ilinov` и `departments`, связанные отношением внешнего ключа: столбец `department_id` таблицы сотрудников ссылается на первичный ключ `id` таблицы отделов.

Обработчик `JoinHandler` выполняет валидацию и собирает итоговый запрос (листинг 31).

```
var selectItems []string
for _, c := range cols1 {
    if r.FormValue("sel1_"+c.Name) == "1" {
        item := sql.SelectAs("t1", c.Name, t1+"."+c.Name)
        selectItems = append(selectItems, item)
    }
}
for _, c := range cols2 {
    if r.FormValue("sel2_"+c.Name) == "1" {
        item := sql.SelectAs("t2", c.Name, t2+"."+c.Name)
        selectItems = append(selectItems, item)
    }
}
if len(selectItems) == 0 {
    selectItems = append(selectItems, "t1.*", "t2.*")
}

querySQL, err := sql.Join(t1, t2, col1, col2, selectItems)
if err != nil {
    data.Status = "error"
    data.Message = "Failed to render JOIN query: " + err.Error()
    return views.Render(w, "join", data)
}

rows, err := pool.Query(querySQL)
if err != nil {
    data.Status = "error"
    data.Message = "Failed to execute INNER JOIN: " + err.Error()
    data.SQL = querySQL
    return views.Render(w, "join", data)
}
defer rows.Close()
```

Листинг 31 — Сборка и исполнение команды `INNER JOIN` в `handlers_query.go`

3.6 Механизм сброса и начальной инициализации схемы

Поскольку лабораторная работа предполагает активное тестирование деструктивных операций DDL и DML, в приложении предусмотрен механизм изоляции и быстрого восстановления исходного состояния базы данных.

3.6.1 Начальная инициализация схемы базы данных

При старте сервера функция `main` вызывает процедуру `initDatabaseTables` (листинг 32).

```
295 func initDatabaseTables(pool *pgx.ConnPool, sql *SQLManager) error {
296     if _, err := pool.Exec(sql.Get("schema.sql")); err != nil {
297         return fmt.Errorf("failed to execute schema SQL: %w", err)
298     }
299
300     var deptCount int
301     if err :=
      ↪  pool.QueryRow(sql.Get("count_departments.sql")).Scan(&deptCount);
      ↪  err != nil {
302         return err
303     }
304
305     if deptCount == 0 {
306         if _, err := pool.Exec(sql.Get("seed_departments.sql")); err != nil
          ↪  {
307             return fmt.Errorf("failed to seed departments: %w", err)
308         }
309     }
310
311     var empCount int
312     if err :=
      ↪  pool.QueryRow(sql.Get("count_employees.sql")).Scan(&empCount); err
      ↪  != nil {
313         return err
314     }
315
316     if empCount == 0 {
317         if _, err := pool.Exec(sql.Get("seed_employees.sql")); err != nil {
318             return fmt.Errorf("failed to seed employees: %w", err)
319         }
320     }
321
322     return nil
323 }
```

Листинг 32 — Инициализация таблиц и первичное наполнение данными в `db.go`

Исходные скрипты встроены в бинарный исполняемый файл с помощью директивы `//go:embed`. Функция выполняет следующие шаги:

а) Создает таблицы `departments` и `lab2_employee_ilinov` с помощью скрипта `schema.sql` (если они еще не созданы);

б) Выполняет проверку количества строк (`SELECT COUNT(*)`) в каждой из таблиц;

в) При отсутствии записей наполняет таблицу отделов 5 строками (`seed_departments.sql`), а затем связанную по внешнему ключу `department_id` таблицу сотрудников 20 строками (`seed_employees.sql`).

3.6.2 Сброс POST /reset

Кнопка `Reset` в шапке веб-интерфейса отправляет запрос на эндпоинт `POST /reset`, активируя обработчик `ResetHandler` (листинг 33).

```
42 func ResetHandler(views *ViewRenderer, pool *pgx.ConnPool, sql *SQLManager)
   ↪ AppHandler {
43     return func(w http.ResponseWriter, r *http.Request) error {
44         if err := resetDatabaseTables(pool, sql); err != nil {
45             base, bErr := newBasePage(pool, sql)
46             if bErr == nil {
47                 base.Status = "error"
48                 base.Message = "Database reset failed: " + err.Error()
49                 return views.RenderStatus(w, "tables",
                   ↪ http.StatusInternalServerError,
                   ↪ TablesPageData{BasePage: base, ShowViewAll: true})
50             }
51             return &AppError{Err: err, Message: "Database reset failed: " +
                ↪ err.Error(), Code: http.StatusInternalServerError}
52         }
53         redirectSuccess(w, r, "/tables")
54         return nil
55     }
56 }
```

Листинг 33 — Обработчик сброса базы данных в `handlers_tables.go`

Обработчик вызывает функцию `resetDatabaseTables` (листинг 34).

```
325 func resetDatabaseTables(pool *pgx.ConnPool, sql *SQLManager) error {
326     tables, err := getTables(pool, sql)
327     if err != nil {
328         return err
329     }
330     for _, t := range tables {
331         dropSQL, err := sql.DropTableIfExists(t)
332         if err != nil {
333             return err
334         }
335         _, _ = pool.Exec(dropSQL)
336     }
337     return initDatabaseTables(pool, sql)
338 }
```

Листинг 34 — Каскадное удаление всех таблиц и повторная инициализация

3.7 Результаты работы и снимки экрана приложения

Для подтверждения корректности работы разработанного веб-сервера и взаимодействия с СУБД PostgreSQL было проведено ручное тестирование всех реализованных сценариев в браузере.

Снимок экрана страницы просмотра таблиц схемы представлен на рисунке 3.1.

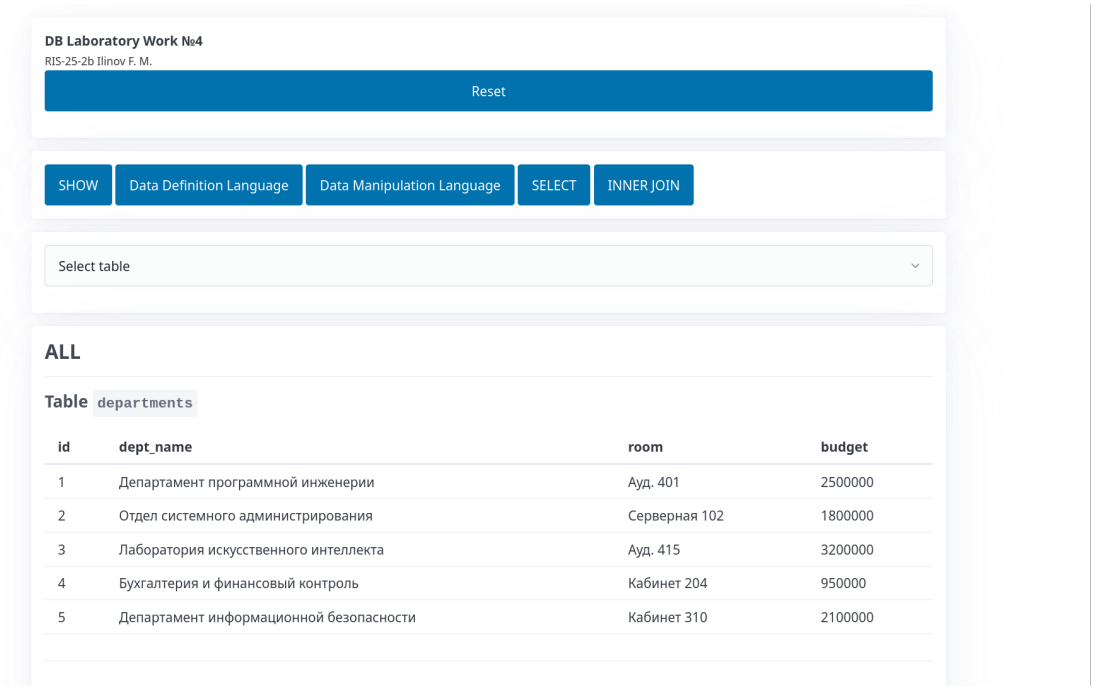


Рисунок 3.1 — Отображение списка таблиц и их кортежей в браузере

Снимок экрана создания новой таблицы с первичным и внешним ключами приведен на рисунке 3.2.

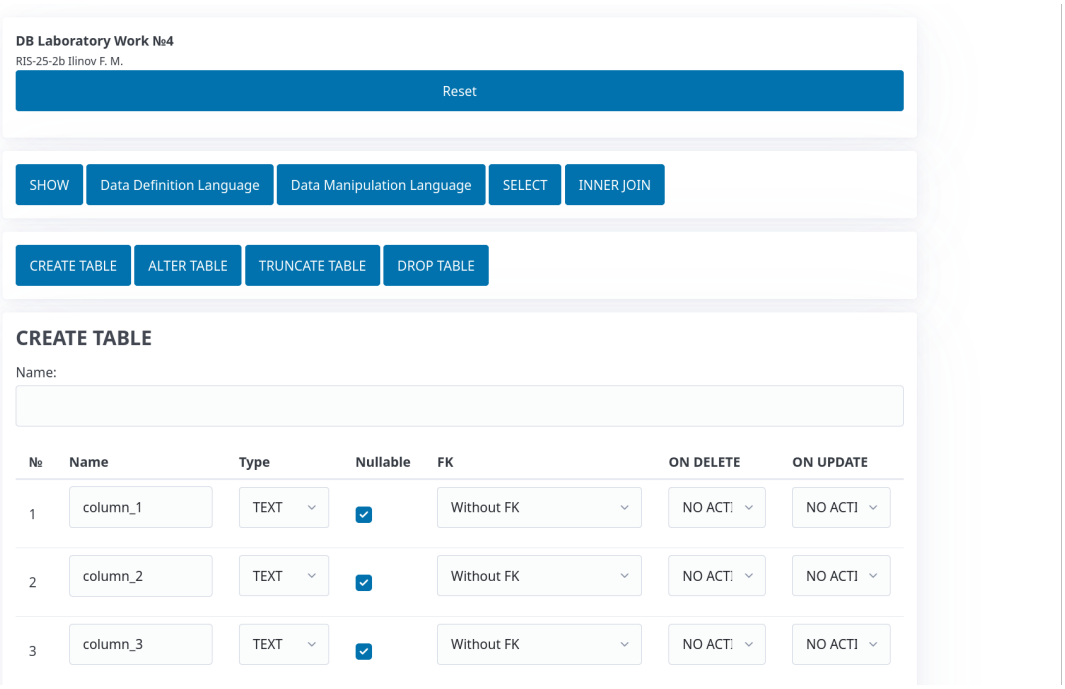


Рисунок 3.2 — Создание таблицы через двухэтапную форму DDL

Снимок экрана модификации колонок таблицы с помощью команды ALTER TABLE приведен на рисунке 3.3.

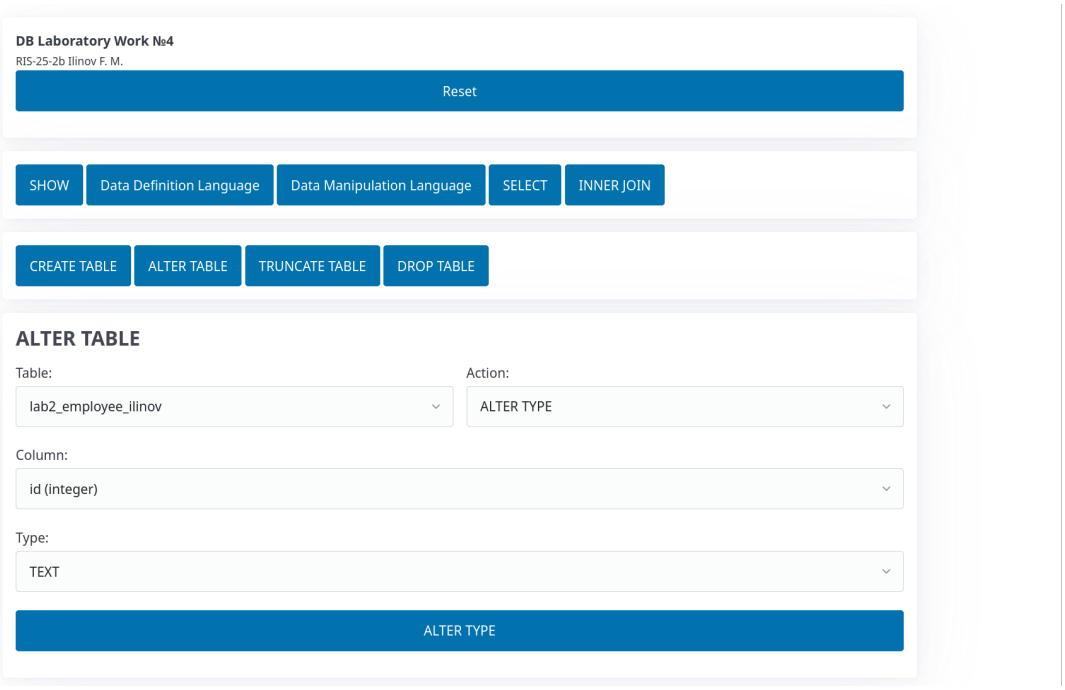


Рисунок 3.3 — Модификация структуры таблицы с помощью команды ALTER TABLE

Снимок экрана очистки данных таблицы оператором TRUNCATE представлен на рисунке 3.4.

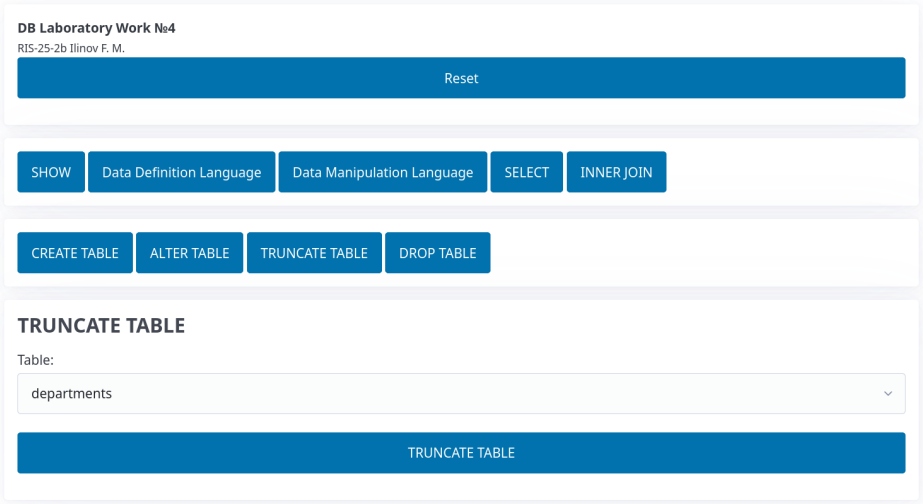


Рисунок 3.4 — Очистка таблицы оператором TRUNCATE TABLE CASCADE

Снимок экрана удаления таблицы с каскадной очисткой связей приведен на рисунке 3.5.

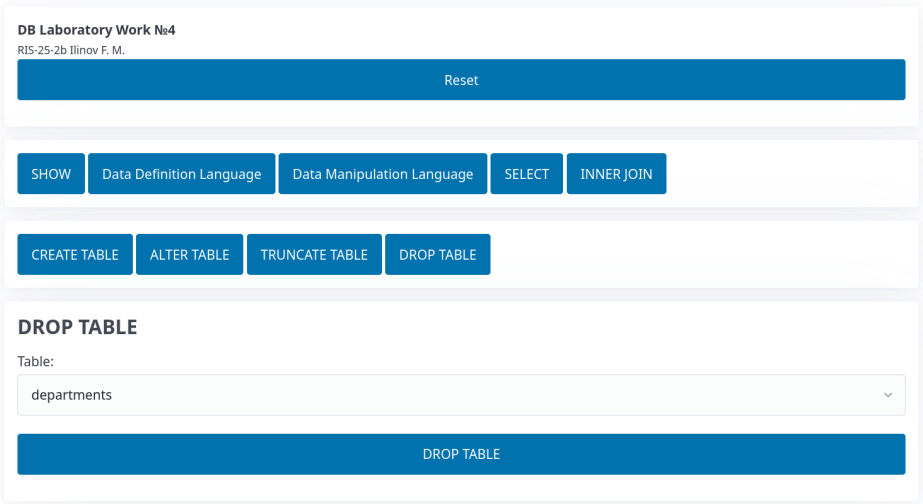


Рисунок 3.5 — Удаление сущности оператором DROP TABLE CASCADE

Снимок экрана пакетного добавления нескольких кортежей оператором INSERT INTO представлен на рисунке 3.6.

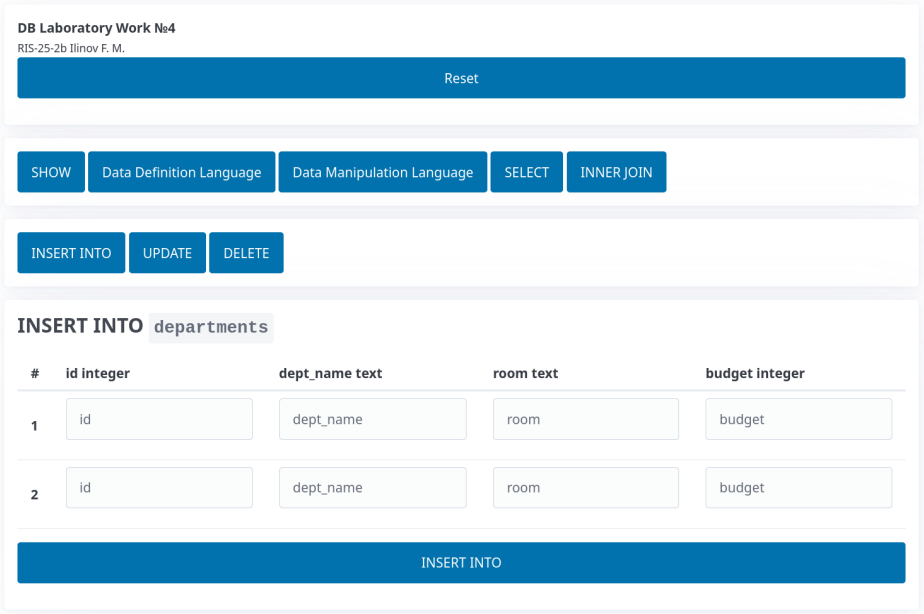


Рисунок 3.6 — Пакетная вставка записей с помощью команды INSERT INTO

Снимок экрана изменения данных оператором UPDATE с предикатом WHERE приведен на рисунке 3.7.

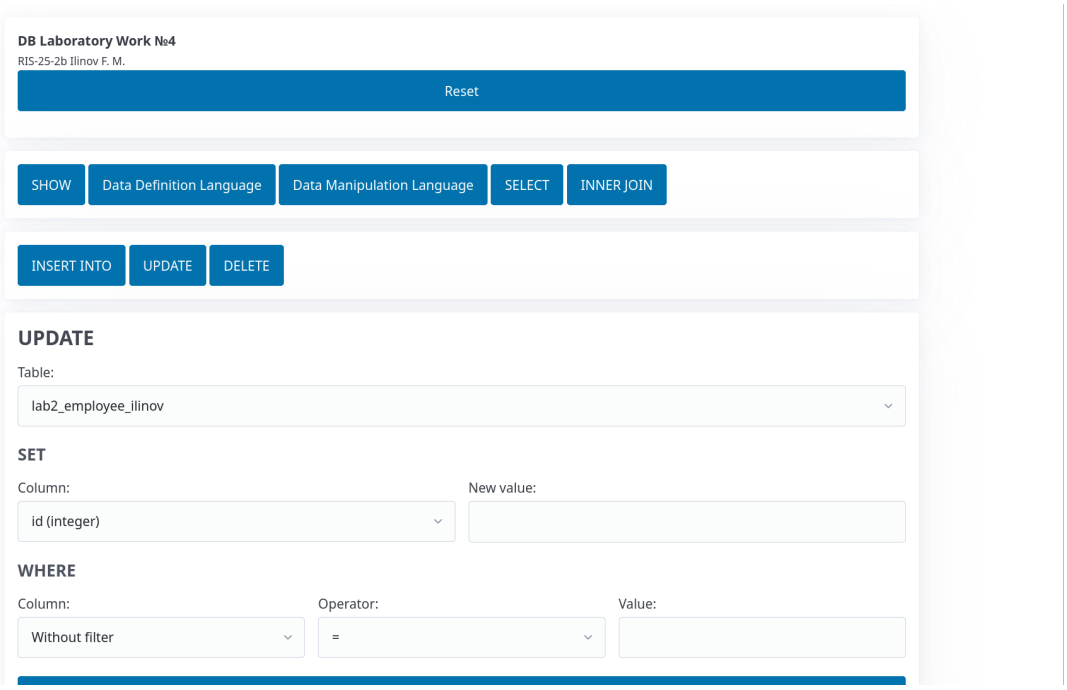


Рисунок 3.7 — Обновление данных с условием фильтрации

Снимок экрана удаления записей оператором DELETE приведен на рисунке 3.8.

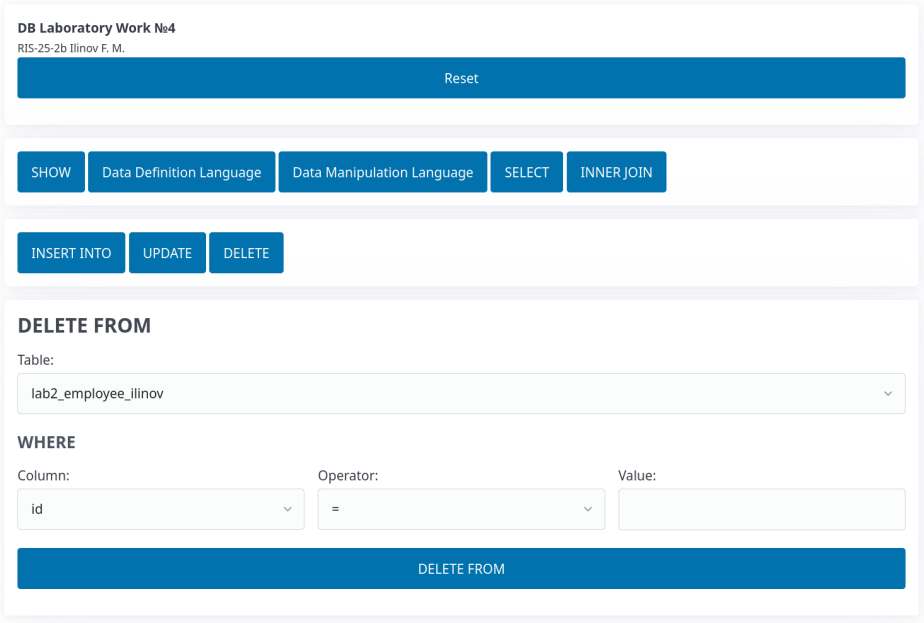


Рисунок 3.8 — Удаление записей оператором DELETE FROM по условию

Снимок экрана параметризованной выборки SELECT с фильтрацией и сортировкой приведен на рисунке 3.9.

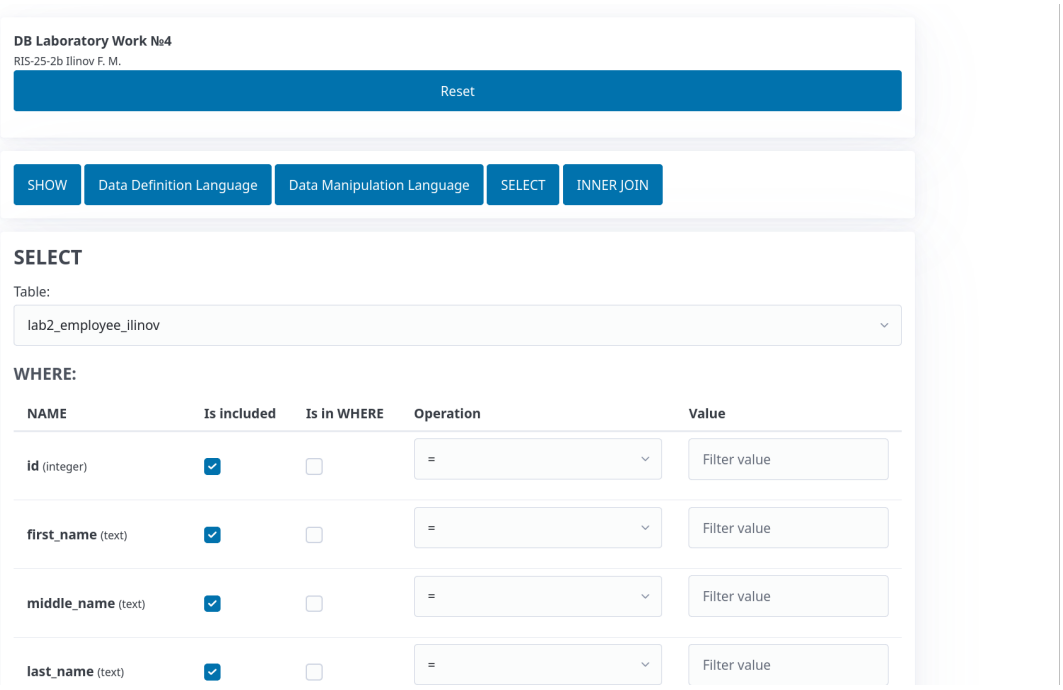


Рисунок 3.9 — Результаты параметризованной выборки SELECT ... WHERE

Снимок экрана внутреннего соединения двух таблиц оператором INNER JOIN представлен на рисунке 3.10.

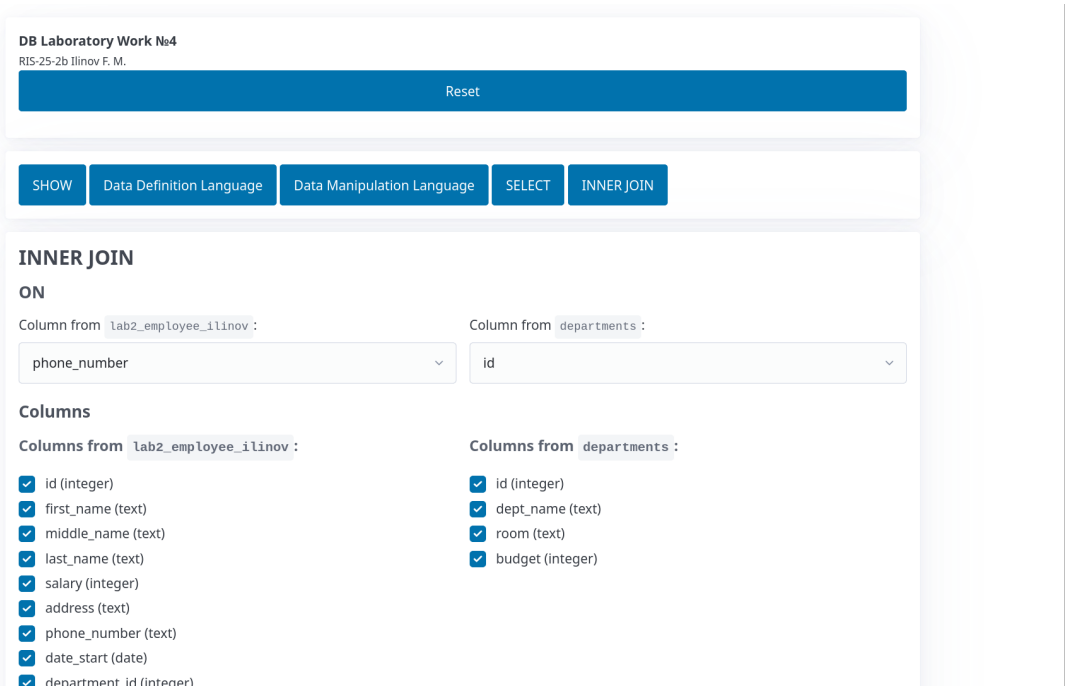


Рисунок 3.10 — Результаты реляционного соединения INNER JOIN ... ON

На всех этапах тестирования сформированные SQL-запросы корректно валидировались и исполнялись СУБД PostgreSQL, а результирующие наборы данных и системные уведомления в полном объеме выводились в веб-интерфейсе.

ЗАКЛЮЧЕНИЕ

В ходе работы были решены следующие задачи:

а) Разработан веб-сервер с модульной архитектурой и сквозным стеком вызовов от пользовательских HTML-форм до ядра СУБД PostgreSQL;

б) Реализовано безопасное конструирование SQL-запросов с экранированием идентификаторов (`quoteIdent`) и литералов (`quoteLiteral`), а также проверкой типов данных, операторов сравнения и правил внешних ключей по белым спискам;

в) Реализована поддержка операций языка определения данных DDL: двухэтапное создание таблиц с поддержкой первичных и внешних ключей (включая самоссылающиеся ограничения), изменение структуры через ALTER TABLE, очистка через TRUNCATE и каскадное удаление через DROP TABLE CASCADE;

г) Реализована поддержка операций языка манипулирования данными DML: пакетная вставка записей с автоматической обработкой NULL-значений, изменение данных по условию UPDATE и удаление данных DELETE;

д) Реализован интерактивный конструктор запросов выборки SELECT с динамической проекцией, фильтрацией и сортировкой, а также оператор реляционного соединения INNER JOIN двух таблиц по общему ключу;

е) Реализован механизм изоляции и быстрого восстановления исходного состояния базы данных через эндпоинт `POST /reset`.

Корректность функционирования подтверждена ручным тестированием в браузере с выводом текста сформированных SQL-команд.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. PostgreSQL 18.6 Documentation. The PostgreSQL Global Development Group. — 2026. — Режим доступа: https://www.postgresql.org/files/documentation/pdf/18/postgresql-18-A4.pdf (дата обращения: 01.09.2026).

2. ISO/IEC. ISO/IEC 9075:2023 Information technology — Database languages — SQL. — 2023. — Режим доступа: https://www.iso.org/standard/76583.html (дата обращения: 01.09.2026).

3. Mozilla. HTTP: Hypertext Transfer Protocol. — 2026. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/HTTP (дата обращения: 01.09.2026).

4. Mozilla. HTML: HyperText Markup Language. — 2026. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/HTML (дата обращения: 01.09.2026).

5. Larroche Lucas. Pico CSS • Minimal CSS Framework for semantic HTML. — 2026. — Режим доступа: https://picocss.com/ (дата обращения: 01.09.2026).

6. Google. The Go Programming Language (version 1.27). — 2026. — Режим доступа: https://go.dev/ (дата обращения: 01.09.2026).

7. jackc. pgx - PostgreSQL Driver and Toolkit. — 2026. — Режим доступа: https://github.com/jackc/pgx (дата обращения: 01.09.2026).